

Output Consistency & Quality System

"AI slop" isn't a mystery — it's what happens when the human gives AI too little to work with. This deliverable encodes the standard once and checks against it twice, before anything ships.

Status v1 · all four skills built, team onboarding next Built by Chii / Chiibitsu Labs

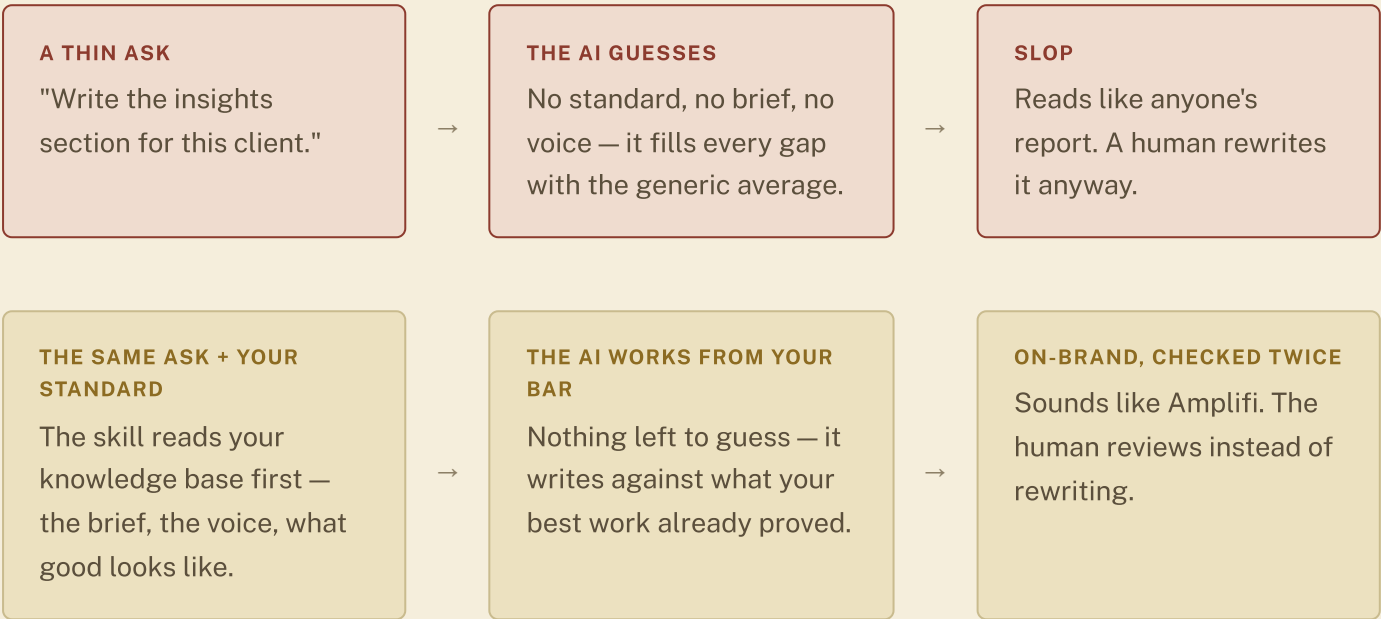
Runs on Every analyst, as they work

The root cause

Generic AI output reads generic because nothing told it not to. Direction is the real variable: give an AI less input, less instruction, and it fills the gap with a guess. Give it real, explicit direction, and it has something to actually work from.

Treating AI as a reason to think less is exactly how slop happens — and how the thinking skill itself atrophies. This deliverable is built on the opposite bet: using AI is a reason to be *more* intentional, not less. Encode the standard explicitly, and "good" stops being a guess for the AI, or for whoever's checking its work.

Same AI. Different input.



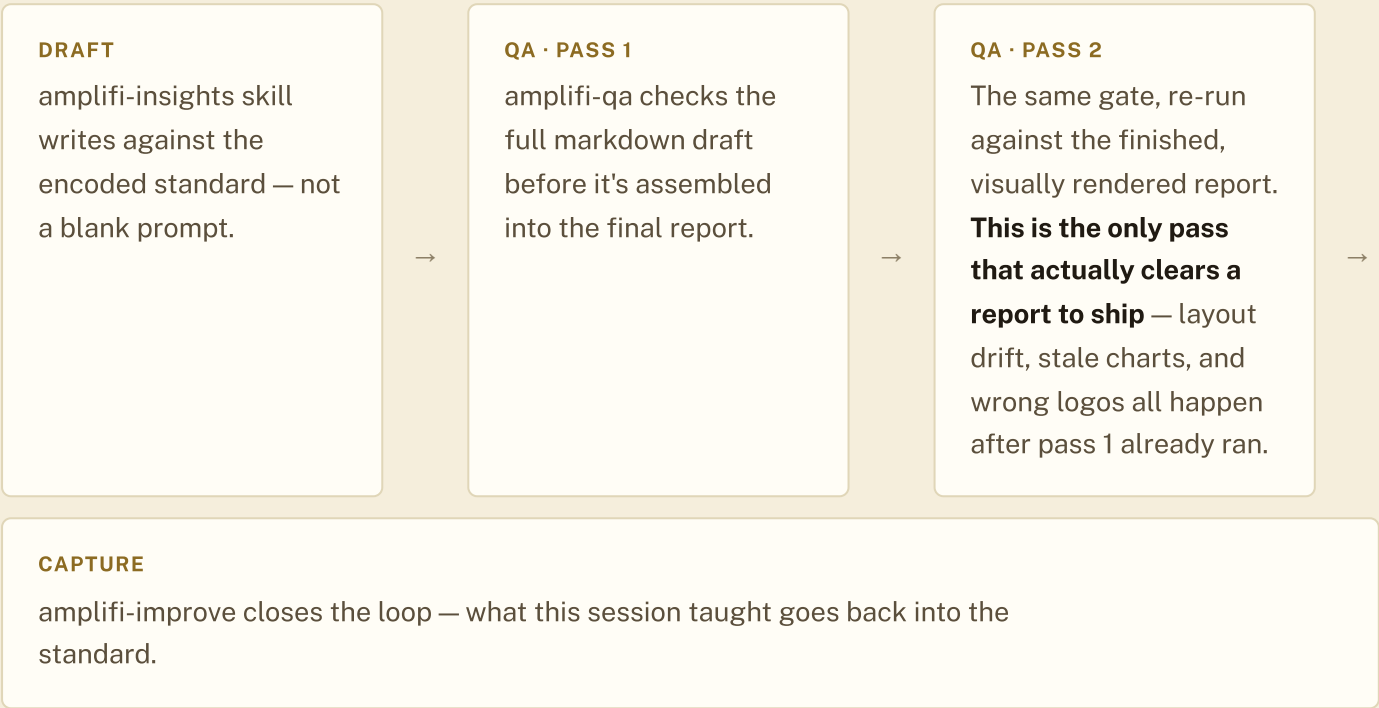
Built just-in-time, not as homework

No one sits down to write four documentation files before they're allowed to start. Every analyst runs one onboarding pass on Claude — it installs the three skills below, and pulls their own function's gold-standard files straight from their own real, already-shipped work. The standard writes itself as a byproduct of work that already proved it, not a project someone has to schedule.

FILE	ANSWERS	WHERE IT COMES FROM
what-good-looks-like.md	What does a strong report actually contain?	Your own best past reports
house-voice.md	How does Amplifi actually sound?	Same reports
report-template-rules.md	What does the template require, structurally?	The live template + brand kit
brand-standard.md PER CLIENT	What does this client expect?	That client's kickoff material

Living documents, not a set of files written once and forgotten — versioned with a changelog, sharpened every time the capture loop feeds something new back in.

Three mechanisms, one pipeline



Why two QA passes, not one

A markdown draft can pass every check and still ship with a stale chart or an off-brand layout once it's actually assembled into the final deck — because that assembly step happens *after* the first check runs. The second pass, against the rendered report, is the one checkpoint that's allowed to actually clear a report for the client. Anything it flags gets a human clear-or-rerun decision, every time.

Argue with the draft before shipping it

One more habit, taught during the sprint: before a report ships, ask what the worst version of this would look like, or have a second pass actively try to find what's wrong — not just confirm it looks fine. A same-angle re-check tends to agree with itself; a pass built to disagree catches what the first one won't. A genuinely different model makes that disagreement more likely than asking the same one to double-check itself — ChatGPT auditing a draft written in Claude, for instance, since that's the tool most people already have open.

The loop-closer

amplifi-improve is what keeps this from calcifying. It's the same skill that runs Deliverable 1's capture loop — every session's real lessons get written down, and the good ones get promoted into these standards files, so the definition of "good" keeps sharpening instead of going stale the day it was written.

Status

All four skills are built and ready — that's done on our side.

amplifi-insights drafts

amplifi-qa checks, twice

amplifi-improve captures

amplifi-onboarding installs the other three and fills the standards live, from real work — nobody's homework

That install is the top of Phase 1 on the roadmap. From there, it's on your team to actually use it day to day — that's the only part left.

